

Expense Tracker

Serverless Cloud Project



[illegible]

The project is built using a serverless 2-tier architecture on AWS, ensuring scalability, low cost, and zero server management.

[illegible]

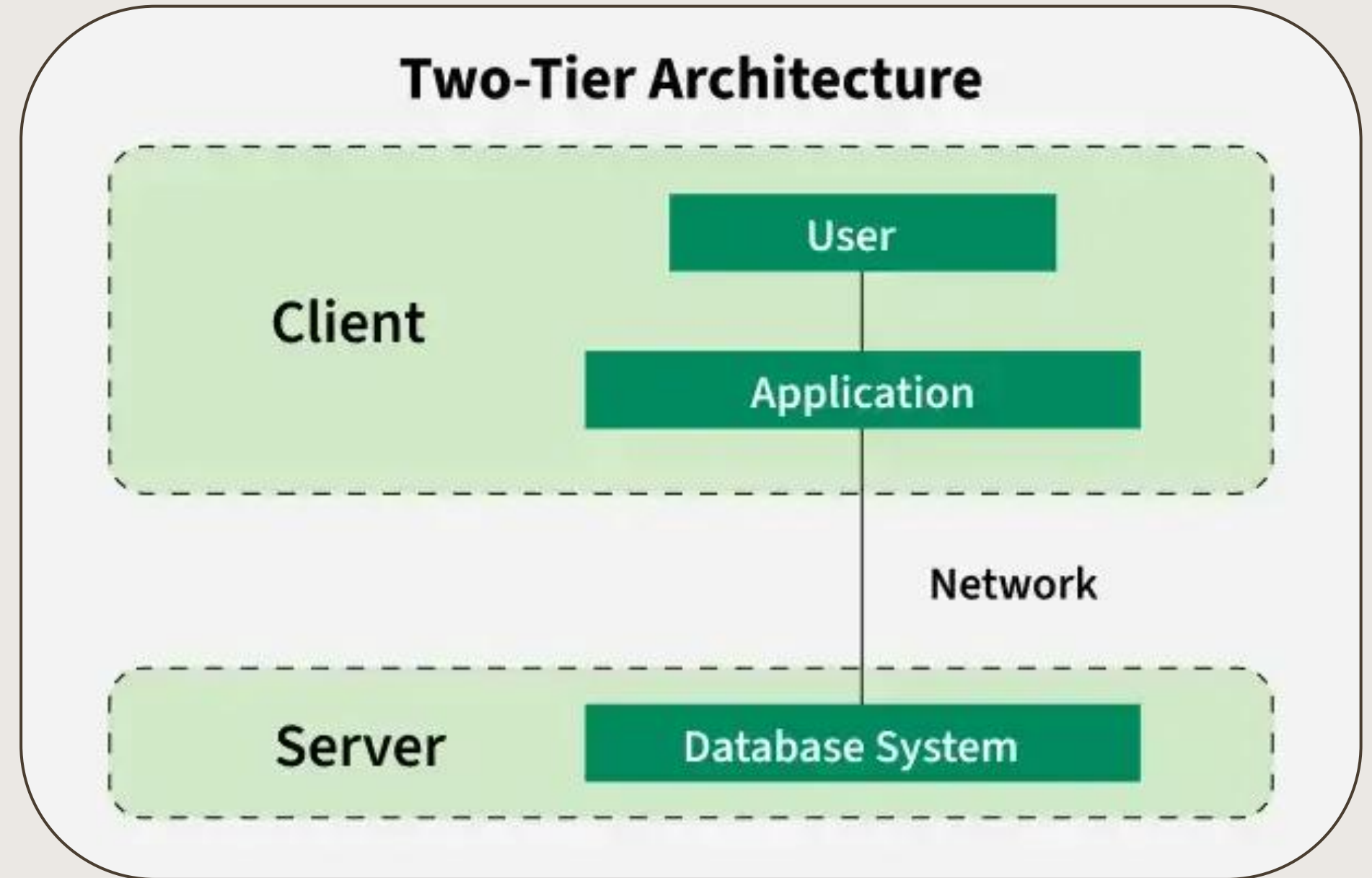
WHAT IS 2-TIER ARCHITECTURE?

A 2-tier architecture separates the application into:

- Frontend Tier – User Interface
- Backend Tier – Business logic and database

In this project:

- Frontend communicates directly with backend APIs
- Backend handles processing and data storage



TECHNOLOGIES USED



Frontend:

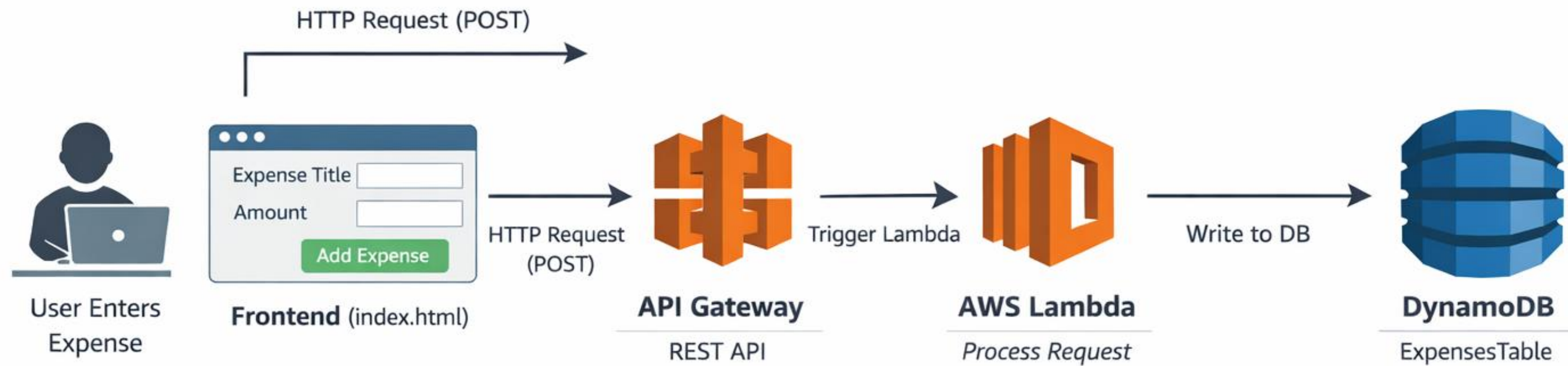
- HTML
- JavaScript

Backend (AWS Services):

- AWS Lambda
- Amazon API Gateway
- Amazon DynamoDB

Other:

- IAM Roles



PROJECT ARCHITECTURE

Flow of the Application

1. User enters expense in web page
2. JavaScript sends request to API Gateway
3. API Gateway triggers Lambda function
4. Lambda stores data in DynamoDB
5. Response is sent back to user

FRONT END

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna ut aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor reprehenderit voluptate velit esse cillum dolore eu fugiat.

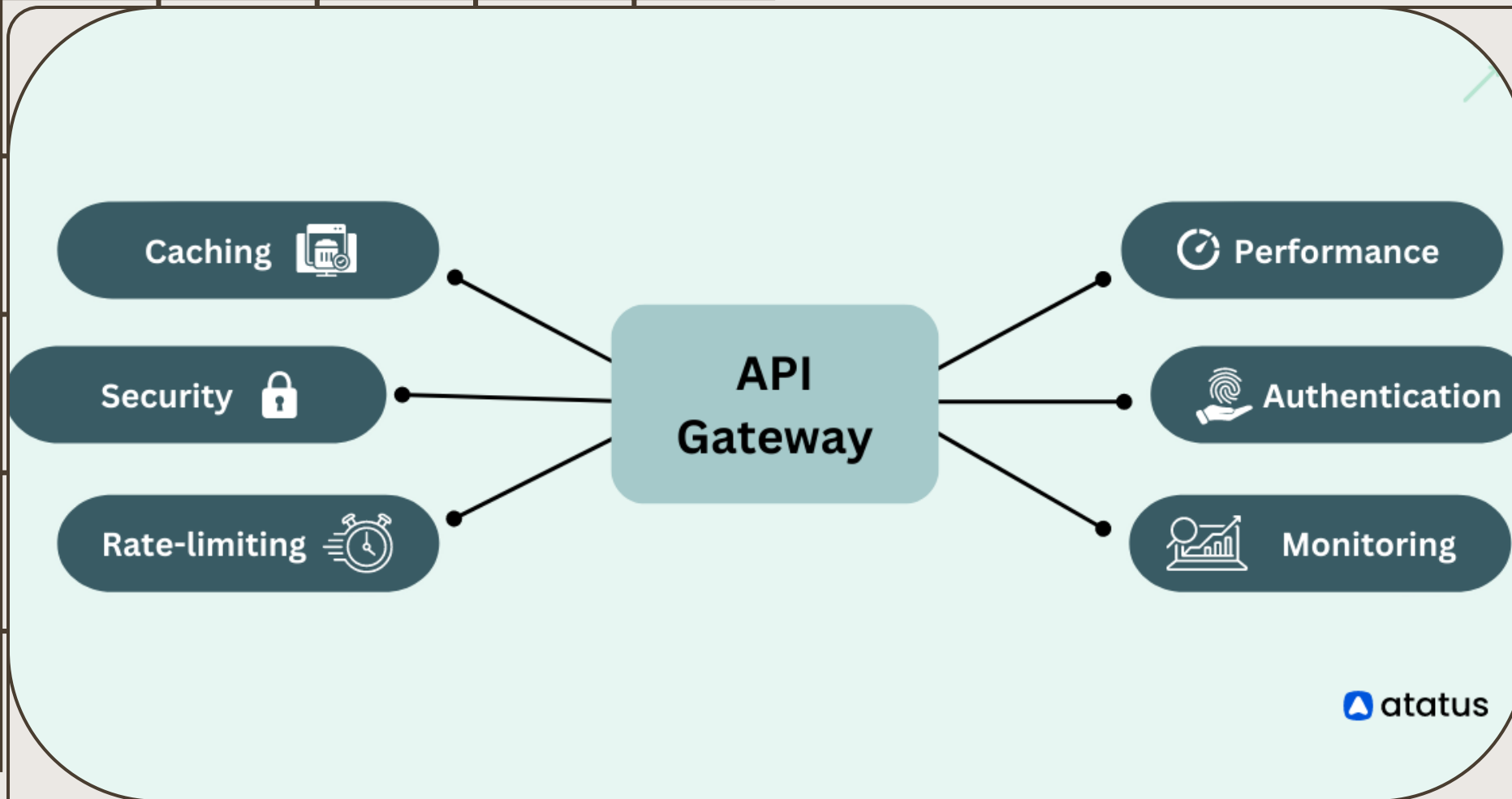


FRONTEND TIER

The frontend is a simple HTML web page that:

- Takes expense title and amount as input
- Uses JavaScript fetch() to call AWS API
- Runs directly in the browser

No server is required for frontend execution.



API GATEWAY

Amazon API Gateway is used to:

- Create REST APIs
- Expose backend Lambda functions as HTTP endpoints
- Handle HTTP methods like GET and POST
- Enable CORS for browser access



aws Lambda

**AWS
LAMBDA**

AWS Lambda is used as the backend logic layer:

- Written in Python
- Handles API requests
- Generates unique expense IDs
- Performs database operations
- Fully serverless and event-driven

AMAZON DYNAMODB

Amazon DynamoDB is used as the database:

- NoSQL key-value database
- Highly scalable and managed
- Stores expense details such as:
 - Expense ID
 - Title
 - Amount
 - Category
 - Date



DynamoDB

IAM ROLE & SECURITY

IAM Role is created to allow Lambda:

- Access DynamoDB
- Write logs to CloudWatch

Security is handled using:

- IAM permissions
- Least privilege access model



CORS CONFIGURATION

CORS (Cross-Origin Resource Sharing) is enabled to:

- Allow browser-based frontend to call API Gateway
- Support GET, POST, and OPTIONS methods
- Prevent browser security blocking

CORS is enabled at the API Gateway resource level.

PROJECT FUNCTIONALITIES



The application supports:

- Add new expense (POST)
- View expenses (GET)
- Serverless execution
- Real-time data storage in DynamoDB

The system works without any server management.

TESTING & VERIFICATION



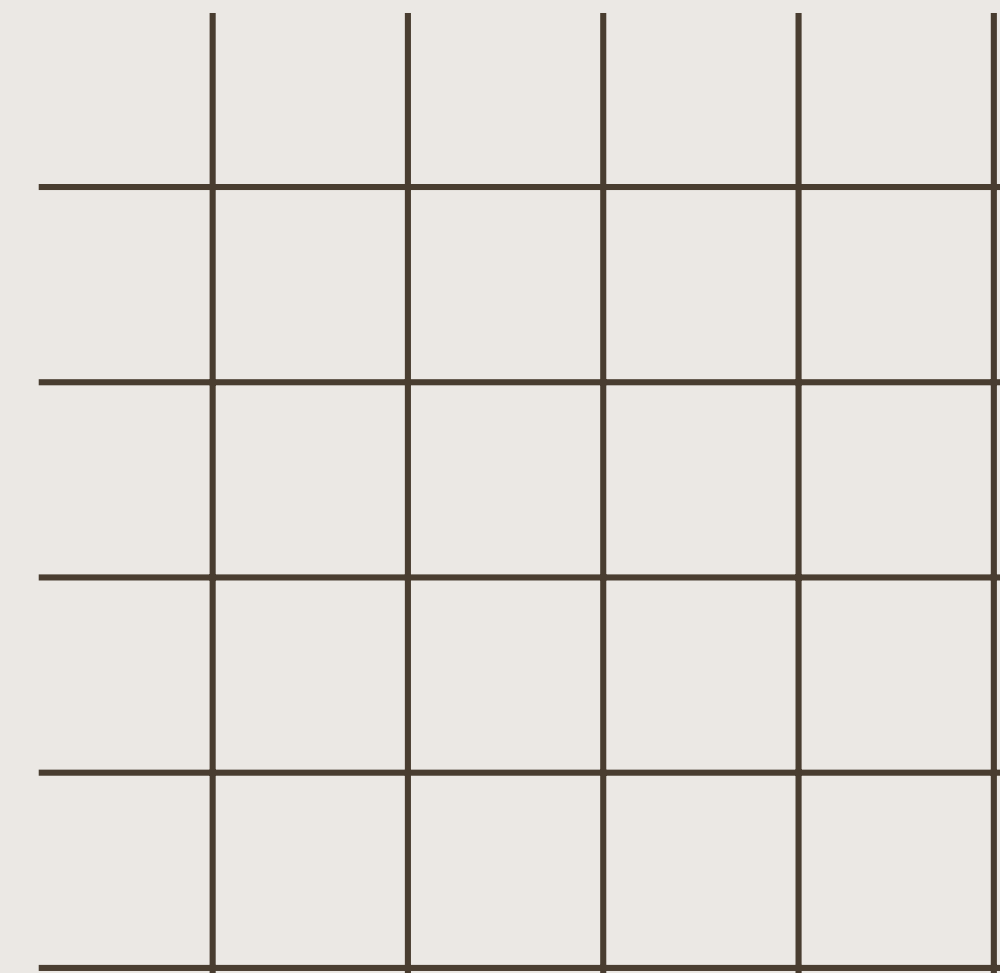
Project testing includes:

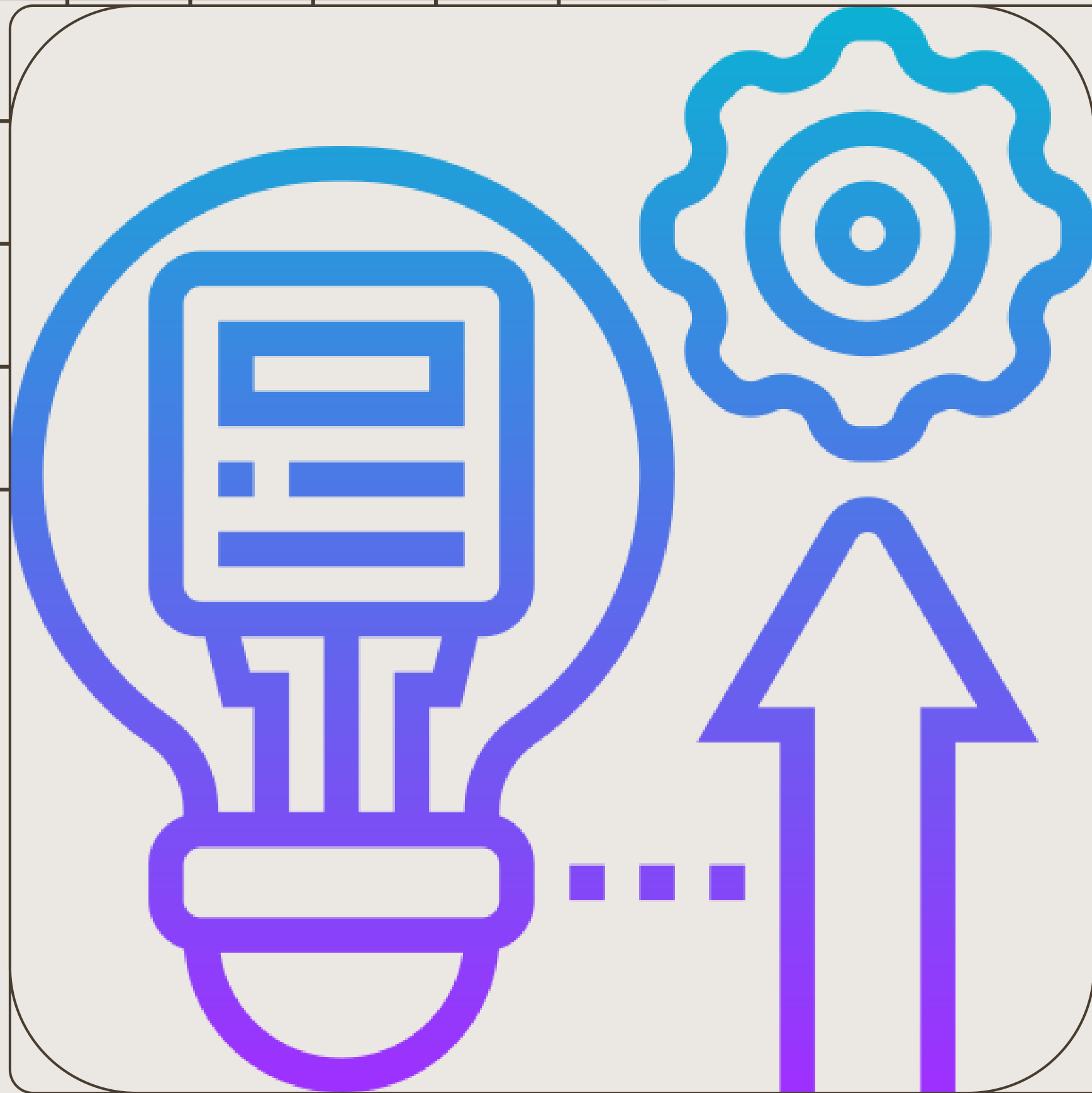
- Testing API using browser
- Adding expense from web page
- Verifying data in DynamoDB table
- Checking logs in CloudWatch

Successful tests confirm end-to-end functionality.



ADVANTAGES OF THIS PROJECT

- Fully serverless architecture
 - Low cost and high scalability
 - No EC2 or server maintenance
 - Beginner-friendly and resume-ready
 - Real-world cloud implementation
- 



Future Enhancements

- Total Expense View
 - Display total expenses (daily / monthly / yearly) on the dashboard
- Expense Visualization
 - Show expenses using simple charts and graphs
- User Authentication
 - Add login system using AWS Cognito for secure access
- Cloud Hosting & Optimization
 - Host frontend on Amazon S3 with CloudFront for better performance

Thank You

